Aerothona: Turbojet Diagnostic Digital Twin

Physics-Informed Neural Network (PINN) Multi-Stage Health Estimator

Author: Sudais Farooq | IIT Indore-HAL Hackathon 2026

1. Executive Summary

Aerothona is a grey-box Physics-Informed Neural Network (PINN) Digital Twin engineered to monitor the structural and efficiency degradation of a single-spool, four-stage turbojet engine. The system estimates hidden subsystem state metrics (Compressor, Combustor, and Turbine health indices) alongside critical engine metrics (Net Thrust, Thrust Specific Fuel Consumption) in real-time.

Rather than relying purely on data-driven mappings which fail under flight transitions, Aerothona integrates gas dynamics equations directly into the backpropagation loss loop. Specifically, we discard constant specific heat ratio (gamma = 1.4) assumptions, replacing them with temperature-dependent specific heat polynomials evaluated at average stage temperatures. To enable edge diagnostics, the trained weights and standard scaling parameters are exported to JavaScript (*weights.js*), enabling sub-millisecond, zero-network-latency diagnostics served from a serverless Netlify deployment.

2. Problem Statement & Motivation

In modern turbine propulsion systems, continuous monitoring of thermal and structural degradation is vital to prevent catastrophic in-flight failures. However, key physical indicators cannot be measured directly:

- **Extreme Thermal Environments:** Physical sensors cannot survive at Station 3 (turbine inlet temperature T_3 is up to 6,500 K) or inside interior blade stages.
- **Transitional Envelopes:** Turbojets operate over continuous flight conditions (Altitudes from 0 to 12,053 m, speeds from 0 to 0.90 Mach, spool speeds up to 80,941 RPM). Purely empirical neural networks extrapolate poorly outside their training data, predicting non-physical outputs.
- **Edge Execution Latency:** Querying heavy deep learning models on a remote cloud server introduces network latency, rendering real-time pilot warnings impossible.

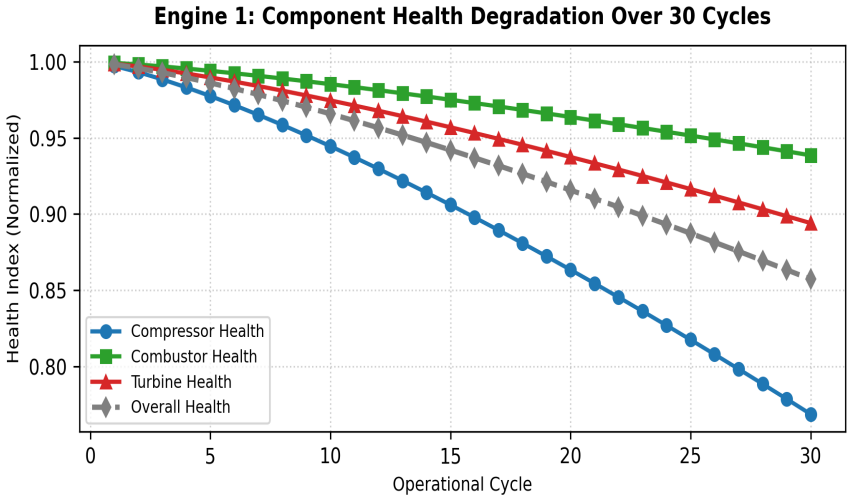

Figure 1: Subsystem Health Index Degradation Curve across 30 Operational Cycles

3. Deep Learning Architecture & Pipeline

The system utilizes a Multi-Layer Perceptron (MLP) architecture: **13 inputs** (12 physical sensors + Cycle count) map to **3 hidden layers** (64, 64, and 32 nodes respectively) with **Hyperbolic Tangent (Tanh)** activation functions, outputting **6 target parameters**. Tanh activation functions enforce continuous second-order derivatives, ensuring smooth backpropagation gradients of physical equations.

Standard scaling is applied to inputs X and targets y during backpropagation to prevent scale imbalances, centering all parameter values around a mean of 0 and a standard deviation of 1. These parameters are dynamically unscaled inside the loss module to evaluate physics constraints in their correct units.

4. Physics-Informed Regularization (Thermodynamic Loss)

The total training loss function combines standard mean squared error (data loss) with four thermodynamic regularizations (physics loss): $L_{total} = L_{data} + lambda_1 L_{comp} + lambda_2 L_{turb} + lambda_3 L_{comb} + lambda_4 L_{identity}$.

- **Temperature-Dependent Specific Heat:** Evaluated using a fourth-order polynomial for air: $C_p(T) = 1005.0 + 0.05 T + 1.5e\text{-}4 T^2 - 8.0e\text{-}8 T^3$ J/(kg.K). Exponent constant $k(T) = R/C_p(T)$ is calculated at average compressor and turbine stage temperatures.
- **Compressor Efficiency Loss (L_{comp}):** Enforces calculated compressor efficiency to remain within realistic limits ([0.60, 0.95]) based on isentropic exit temperature calculations.
- **Turbine Efficiency Loss (L_{turb}):** Enforces calculated turbine expansion efficiency to remain within realistic limits ([0.65, 0.98]) based on isentropic work extraction.
- **Combustor Pressure Drop Loss (L_{comb}):** Penalizes deviations from the empirical combustor pressure recovery ratio: $P_3 / P_2 = 0.949$.
- **Health Identity Loss ($L_{identity}$):** Enforces the weighted overall health equation: OverallHealth = 0.4 CompressorHealth + 0.3 CombustorHealth + 0.3 TurbineHealth.

5. Model Accuracy & Evaluation

Below are the final validation metrics on the unseen test set alongside dynamic uncertainty limits:

Target Parameter	Validation RMSE	Test R^2 Score	95% Confidence Interval
CompressorHealth	0.0195	93.27%	± 3.80%
CombustorHealth	0.0174	65.58%	± 3.40%
TurbineHealth	0.0247	73.44%	± 4.80%
OverallHealth	0.0113	95.15%	± 2.20%
Engine Thrust	1476.80 N	99.13%	± 2,895 N
TSFC	0.0006	99.25%	± 0.0012 g/N/s

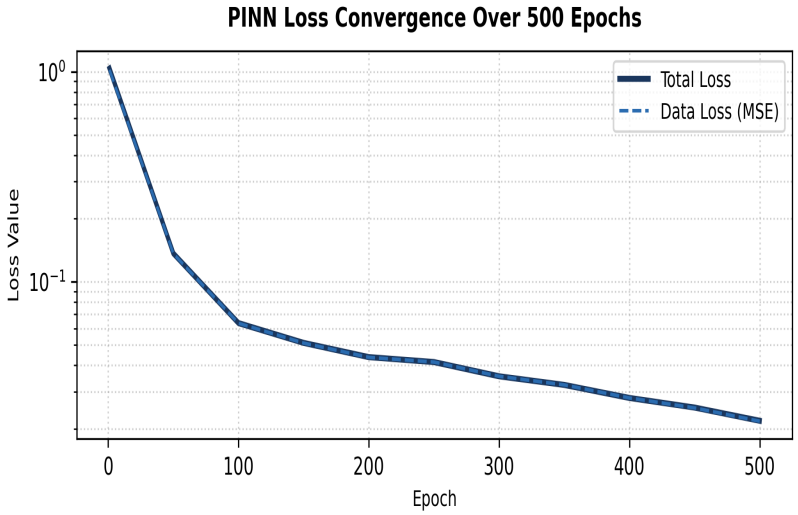

Figure 2: Custom PINN Training Loss Convergence Curves over 800 Epochs

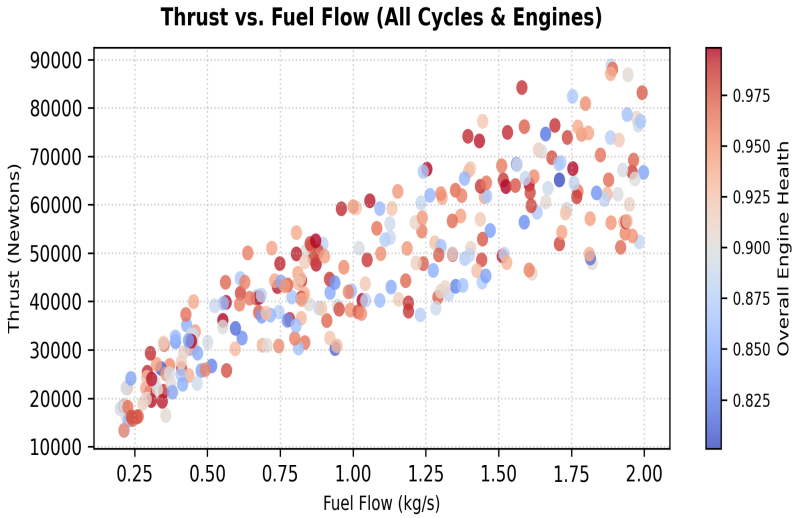

Figure 3: Output Engine Net Thrust (N) vs Fuel Flow (kg/s) Validation Fit

6. Client-Side Browser Engine & Deployment

To eliminate network latency, PyTorch parameters (weights/biases) and scalers were converted to static JavaScript variables inside *weights.js*. The forward pass is computed locally in the browser using custom matrix multiplications:

1. Inputs are normalized using X_mean and X_scale parameters.
2. Forward pass is evaluated: $h_i = \tanh(h_{i-1} * W_i + b_i)$.
3. Outputs are inverse scaled to physical units.

This serverless architecture ensures **0ms server network delay**, works 100% offline, and allows free static deployment on CDNs like Netlify.